



1.1 What Is a Design Pattern?

Christopher Alexander says, "Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice" [AIS⁺77, page x]. Even though Alexander was talking about patterns in buildings and towns, what he says is true about object-oriented design patterns. Our solutions are expressed in terms of objects and interfaces instead of walls

Most asked design patterns in Interviews:

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Table 1.1: Design pattern space

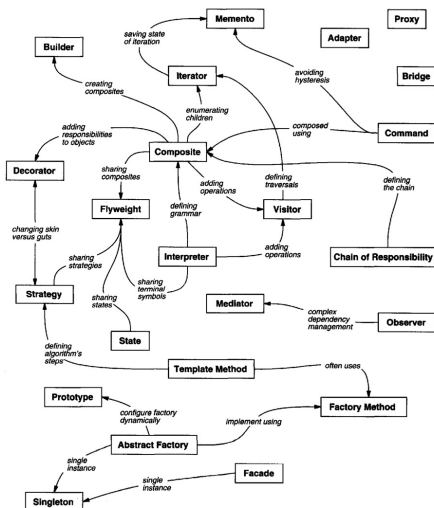


Figure 1.1: Design pattern relationships

CREATIONAL PATTERNS

Factory Design Pattern

⇒ Define an interface for creating an object but let subclasses decide which class to instantiate

When to use :

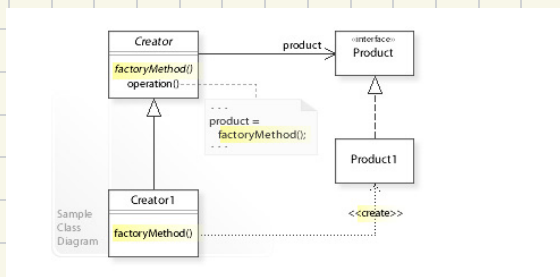
- You find new scattered everywhere for a family of types
- Abstract the concrete class used.
- You plan to add new variants later

Pros of using

- Loose coupling and cleaner.
- Single responsibility for construction logic
- Simple path to add variants easily

Cons :

- Adds extra layer of abstraction
- Inflated number of helper classes.



⇒ Basic UML structure

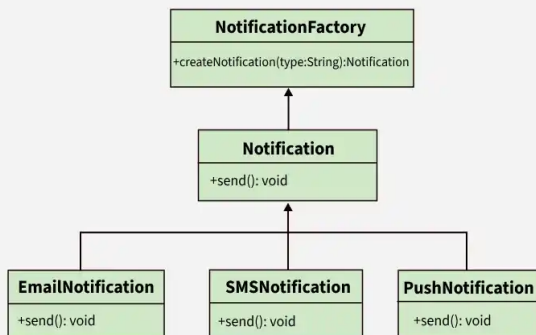
What are creational design patterns?

This design pattern helps make the instantiation process abstract.

- It helps making a system independent from the object creation, how objects are composed or represented.

Analogy to remember

⇒ Imagine using a **COFFEE** machine; now via this machine you have the option to create multiple different types of coffee without being worried about the "method to create all types of drink"; Here coffee machine is our factory class



Simplest example of Factory Pattern